

You are acting as a Principal Product Engineer, Staff React Native Engineer, Full-stack Architect, Design System Engineer, Motion Designer, and AI Product Builder.

Your task is to initialize, plan, architect, and begin implementing a production-ready product named **CHROMAWAVE**.

Do not treat this as a simple UI prototype or hackathon demo.

The goal is to build a polished, scalable, commercially viable application with premium visual quality, strong motion design, reliable architecture, and a clear path from MVP to public beta.

1. Product concept

CHROMAWAVE is a synesthetic memory application that combines:

- Color curation
- Music discovery
- Photography
- Mood-based memory collection
- Social discovery
- Visual storytelling

The core idea:

A user captures a real-world moment through the camera or photo library.

CHROMAWAVE extracts the dominant color palette from that image, analyzes its emotional character, and pairs it with music or a soundscape that evokes a similar feeling.

Each saved item becomes a “Memory” containing:

- Original photo
- Extracted color palette
- Dominant colors
- Mood and vibe tags
- Selected track, playlist, or soundscape
- Timestamp
- Optional location
- Personal note
- Animated visual representation
- Sharing configuration

The product should make everyday moments both visually and audibly memorable.

2. Core experience lifecycle

The main user journey revolves around five stages.

Capture

The user:

- Takes a photo
- Selects an existing photo
- Optionally enters a note
- Optionally attaches location
- Can preview the detected palette before continuing

The app should extract:

- Dominant colors
- Accent colors
- Color percentage distribution
- Brightness
- Saturation
- Temperature
- Contrast
- Overall visual mood

Pair

The system pairs the extracted palette with music.

The user can:

- Accept an automatic suggestion
- Regenerate suggestions
- Search manually
- Preview tracks
- Save a track
- Save a playlist
- Select a generated ambient soundscape in future versions

Pairing recommendations should consider:

- Palette hue
- Brightness
- Saturation
- Temperature
- Contrast

- User music preferences
- Listening history
- Saved memories
- Time of day
- Optional mood input

Collect

The user builds a personal timeline of visual-audio memories.

The library should support:

- Chronological timeline
- Mood grouping
- Color grouping
- Month and year views
- Search
- Filters
- Favorites
- Collections
- Private and public memories
- Animated memory details

Discover

Users can explore public memories from other users.

Discovery should support:

- Browse by palette
- Browse by mood
- Browse by music genre
- Browse by season
- Browse by location when permitted
- Discover tracks through visual aesthetics
- Save inspiration
- Follow creators in a later milestone

Share

Users can export or share:

- Animated palette cards
- Static mood boards
- Memory videos
- Palette and track combinations

- Spotify playlists
- Social story formats
- Printable color cards in the Premium tier

3. Target audience

Primary users:

- Designers
- Visual artists
- Photographers
- Musicians
- Music lovers
- Creative professionals
- Journal users
- People who enjoy aesthetic content
- People who want to preserve memories in a more emotional format

The app must feel creative and expressive without becoming confusing or overly abstract.

4. Product positioning

CHROMAWAVE should not look like:

- A generic photo gallery
- A standard music streaming application
- A basic mood journal
- A simple color picker
- A template-driven social app

It should feel like a fusion of:

- A premium creative tool
- A personal memory journal
- A music discovery product
- A visual art collection
- A living emotional archive

The design should feel distinctive enough to be recognizable from a single screenshot.

5. Platform strategy

Build the product with a mobile-first architecture.

Primary application:

- React Native
- Expo
- TypeScript
- iOS and Android support
- Mobile-first UX

Web expansion:

- Next.js
- Shared domain models
- Shared backend
- Shared design tokens where practical
- Web experience can initially focus on public memory pages, discovery, marketing, account management, and shared collections

Choose a maintainable monorepo structure unless the current repository already has a better architecture.

Recommended direction:

- pnpm
- Turborepo
- Expo Router
- Next.js App Router
- Shared TypeScript packages

Possible structure:

```
apps/  
  mobile/  
  web/  
  
packages/  
  ui/  
  design-tokens/  
  domain/  
  api/  
  database/  
  analytics/  
  config/  
  eslint-config/  
  typescript-config/
```

Do not blindly use this structure if repository constraints suggest a better alternative.

First inspect the workspace and document the architectural decision.

6. Proposed technical stack

Use the following stack as the starting direction, but validate each choice before installing it.

Mobile

- React Native
- Expo
- Expo Router
- TypeScript
- React Native Reanimated
- React Native Gesture Handler
- Expo Image
- Expo Camera
- Expo Image Picker
- Expo Haptics
- Expo Blur
- Expo AV or the current recommended Expo audio solution
- Skia only where it provides meaningful visual or performance benefits

Web

- Next.js
- TypeScript
- App Router
- Server Components where appropriate
- Framer Motion or Motion
- Responsive public pages

State and networking

- Zustand for focused client state
- TanStack Query for server state
- Zod for validation
- React Hook Form for complex forms

Do not duplicate server state in Zustand.

Backend

- Supabase
- PostgreSQL
- Supabase Auth
- Supabase Storage
- Row Level Security
- Edge Functions only where justified

- Realtime only for features that actually require it

Integrations

Initial music integration:

- Spotify

Architecture must allow later support for:

- Apple Music
- YouTube Music
- Generated ambient soundscapes

Create a provider abstraction instead of tightly coupling the domain to Spotify.

Color extraction

Start with an implementation that can reliably extract dominant colors from local images.

Evaluate:

- K-means clustering
- Histogram-based extraction
- Median cut
- Native or WebAssembly-assisted processing
- Server-side processing as a fallback

The first production version should prioritize:

- Perceived quality
- Processing speed
- Memory usage
- Deterministic results
- Mobile compatibility

Use perceptual color spaces such as LAB or OKLCH where they improve clustering and mood analysis.

Do not treat raw RGB Euclidean distance as the final production-quality approach without evaluation.

7. Motion-first design philosophy

Motion is a core interaction system, not decoration.

Required motion concepts:

- Palette colors flow into place after extraction
- Gradients evolve when a Memory loads
- Waveforms respond to audio playback
- Memory cards transition naturally into detail screens
- Timeline scrolling includes restrained depth and parallax
- Palette chips react to touch
- Music pairing transitions feel connected to the selected colors
- Navigation changes preserve visual continuity
- Loading states feel expressive but remain fast
- Haptics reinforce meaningful actions

Motion rules:

- Most direct interaction feedback should complete within 300 milliseconds
- Longer transitions are allowed only for deliberate immersive moments
- Avoid animation that blocks interaction
- Avoid decorative motion overload
- Maintain 60 FPS on normal target devices
- Prefer native-thread animations
- Respect Reduce Motion accessibility settings
- Avoid layout thrashing
- Avoid excessive blur, overdraw, and full-screen animated gradients when performance is compromised

Every implemented animation must have:

- Purpose
- Trigger
- Duration
- Easing or spring configuration
- Reduced-motion behavior
- Performance consideration

8. Visual direction

Create a high-end visual language specifically for CHROMAWAVE.

The interface should feel:

- Emotional
- Chromatic
- Fluid
- Artistic
- Modern
- Tactile

- Immersive
- Premium
- Calm when browsing
- Energetic when playing music

Avoid:

- Generic glassmorphism everywhere
- Excessive bordered cards
- Random gradients
- Inconsistent corner radii
- Heavy shadows on every element
- Template-like dashboards
- Generic purple AI-app styling
- Neon cyberpunk clichés
- Overcrowded screens
- Huge amounts of text
- UI elements without hierarchy

The user-generated palette should influence the UI contextually, but must not destroy:

- Readability
- Contrast
- Brand consistency
- Accessibility
- Navigation recognition

Build a neutral base design system that can safely receive palette-derived accent colors.

9. Accessibility requirements

Target WCAG 2.1 AA where applicable.

Include:

- Sufficient text contrast
- Dynamic type support
- Screen reader labels
- Logical focus order
- Minimum touch target sizes
- Reduced-motion support
- Color-blind-safe states
- Non-color indicators for selection and errors
- Captions or visual alternatives where relevant
- Clear permission explanations
- Accessible audio controls

Do not rely on palette colors alone to communicate state.

10. Subscription model

Implement the architecture so entitlement rules are centralized and cannot become scattered conditional checks across screens.

Free

Price:

- \$0

Limits:

- Up to 10 saved palettes or Memories
- Up to 5 music pairings
- Basic timeline
- Basic sharing
- Limited discovery
- Standard export quality

Pro

Price:

- \$5.99 per month

Includes:

- Unlimited Memories
- Unlimited pairings
- AI-assisted pairing suggestions
- Advanced filters
- High-quality exports
- More collections
- Extended discovery
- Custom mood tags

Premium

Price:

- \$11.99 per month

Includes everything in Pro, plus:

- Printable palettes
- Spotify playlist export
- Premium animated exports
- Print-on-demand color cards
- Exclusive visual themes
- Early access to generated soundscapes
- Premium profile and collection customization

Use a subscription provider abstraction.

For mobile, evaluate RevenueCat as the default subscription orchestration layer.

Do not build fake checkout logic into production application flows.

During early development, implement a typed mock entitlement provider that can later be replaced by RevenueCat.

11. Privacy and permissions

The application processes personal photos, music preferences, optional locations, and personal notes.

Implement privacy by design.

Requirements:

- Request camera permission only when needed
- Request photo access only when needed
- Request location only when explicitly enabled
- Clearly explain why each permission is requested
- Default memories to private
- Require explicit action to publish a Memory
- Strip unnecessary EXIF metadata before public sharing
- Never expose storage paths directly
- Use signed URLs where appropriate
- Protect private media with storage policies
- Use Supabase Row Level Security
- Provide account deletion architecture
- Provide data export architecture
- Avoid uploading photos before user confirmation unless product behavior clearly requires it

Document the threat model and privacy decisions.

12. Core domain model

Design the domain model before building screens.

At minimum, evaluate entities such as:

```
User
UserProfile
Memory
MemoryAsset
Palette
PaletteColor
MoodTag
MusicPairing
MusicTrack
MusicPlaylist
MusicProviderAccount
Collection
CollectionItem
Favorite
PublicMemory
Follow
Subscription
Entitlement
ExportJob
ShareLink
```

A Memory should be the central aggregate.

Do not allow UI components to define the domain model implicitly.

Create:

- Domain types
- Zod schemas
- Database mapping
- API contracts
- Repository interfaces
- Provider interfaces
- Migration strategy

13. Music provider abstraction

Create a provider interface similar to:

```
interface MusicProvider {
  connect(): Promise<void>;
  disconnect(): Promise<void>;
  searchTracks(query: string): Promise<MusicTrack[]>;
  getRecommendations(input: RecommendationInput): Promise<MusicTrack[]>;
  getTrack(id: string): Promise<MusicTrack>;
  getAudioPreview(id: string): Promise<AudioPreview | null>;
  createPlaylist(input: CreatePlaylistInput): Promise<PlaylistResult>;
}
```

The actual interface can differ, but the domain must not depend directly on Spotify response objects.

Spotify-specific models must stay in an infrastructure layer.

Handle the possibility that:

- Preview URLs are unavailable
- User authentication expires
- Market availability differs
- Tracks are unavailable in a region
- Playlist export requires additional scopes
- API quotas apply

14. Pairing engine architecture

The first pairing engine does not need to use a large AI model.

Design an explainable, testable recommendation pipeline.

Potential steps:

1. Extract visual features.
2. Convert colors into perceptual metrics.
3. Classify palette mood.
4. Map mood to audio characteristics.
5. Retrieve candidate tracks.
6. Rank candidates.
7. Apply user preference signals.
8. Add diversity.
9. Return recommendation explanations.

Possible audio features:

- Valence
- Energy

- Tempo
- Acousticness
- Instrumentalness
- Danceability
- Loudness
- Genre
- Time signature

Do not assume all Spotify audio-feature endpoints remain available or unrestricted.

Keep the recommendation engine behind an interface so data sources can change.

The user-facing explanation can be concise, for example:

- "Warm amber tones paired with mellow acoustic energy."
- "High contrast and electric blue tones matched with an energetic electronic track."

15. Analytics strategy

Create an analytics abstraction.

Potential events:

- onboarding_started
- onboarding_completed
- capture_started
- capture_completed
- palette_extracted
- palette_edited
- pairing_requested
- pairing_accepted
- pairing_changed
- memory_saved
- memory_shared
- memory_published
- discovery_opened
- discovery_item_viewed
- track_preview_started
- track_saved
- paywall_viewed
- subscription_started
- export_started
- export_completed

Analytics must not include raw private photo content or personal notes.

Add a typed analytics event map.

16. Testing strategy

Set up testing from the beginning.

Include:

- Unit tests for color utilities
- Unit tests for palette mood classification
- Unit tests for entitlement rules
- Unit tests for pairing ranking
- Schema validation tests
- Repository tests
- Component tests for critical interactions
- Navigation smoke tests
- Basic end-to-end happy path
- Accessibility checks where tooling permits
- Database policy tests
- Integration tests for provider adapters using mocks

Avoid testing implementation details.

17. Observability and resilience

Prepare the app for production debugging.

Include:

- Structured error handling
- Error boundaries
- Typed domain errors
- Logging abstraction
- Crash reporting integration point
- Network retry policy
- Offline and degraded states
- Empty states
- Permission-denied states
- Spotify disconnected state
- Expired session state
- Upload failure recovery
- Color extraction failure fallback

Never leave core screens permanently blocked by a spinner.

18. Offline behavior

The application should remain useful when temporarily offline.

At minimum:

- Users can view already cached private Memories
- Draft captures can be stored locally
- Pending uploads can retry later
- Failed pairings can be retried
- UI clearly distinguishes local drafts from synchronized Memories

Do not promise full offline music playback unless licensing and provider capabilities support it.

19. Development roadmap

Plan the project around approximately 10–13 weeks to beta.

M0 — Foundation

Goals:

- Inspect or initialize repository
- Decide monorepo architecture
- Configure mobile and web apps
- Configure TypeScript
- Configure linting and formatting
- Configure testing
- Create environment strategy
- Create shared domain package
- Create design tokens
- Create foundational UI system
- Create Supabase local setup
- Create CI baseline
- Create architecture documents
- Create development scripts

M1 — Capture and color extraction

Goals:

- Camera flow
- Photo library flow
- Image preview
- Color extraction

- Palette editing
- Mood classification
- Save draft Memory
- Local caching
- Upload architecture
- Capture permissions
- Capture onboarding

M2 — Music integration and pairing

Goals:

- Spotify authentication
- Provider abstraction
- Track search
- Recommendation pipeline
- Track preview when available
- Pairing UI
- Save pairing
- Re-pair Memory
- Spotify error and disconnected states

M3 — Timeline, discovery, and social foundation

Goals:

- Personal timeline
- Memory detail
- Search and filters
- Collections
- Public Memory publishing
- Discovery feed
- Share links
- Basic public web page
- Favorites
- Reporting and moderation architecture

M4 — Monetization and release readiness

Goals:

- Subscription architecture
- RevenueCat integration
- Paywall
- Entitlement enforcement
- Premium exports
- Analytics

- Crash reporting
- Privacy review
- Accessibility review
- Performance review
- Store assets and release checklist
- Beta distribution

Post-launch

Potential work:

- Creator profiles
- Follow system
- Community curation
- Collaborative collections
- Generated ambient soundscapes
- Apple Music
- Print-on-demand
- Seasonal color journeys
- Memory recap
- AI-written memory captions
- Emotion trends
- Brand and artist partnerships

20. Required working method

You must follow this sequence.

Phase 0 — Workspace inspection

Before modifying files:

1. Inspect the entire repository.
2. Identify:
3. Existing apps
4. Package manager
5. Framework versions
6. Build configuration
7. Existing design system
8. Existing backend
9. Existing environment files
10. Existing tests
11. Existing CI

12. Existing conventions
13. Run safe read-only checks.
14. Do not delete or overwrite existing work.
15. Record any blockers or inconsistencies.

16. Determine whether this is:

17. A blank repository

18. A partially initialized repository

19. An existing product being adapted

If the repository is empty, initialize it.

If it already contains a project, adapt the plan to the real codebase.

Do not pretend files or services exist if they are not present.

Phase 1 — Planning and architecture

Before large-scale implementation, create the following documents:

```
docs/  
  00-product-brief.md  
  01-repository-audit.md  
  02-architecture-overview.md  
  03-domain-model.md  
  04-database-design.md  
  05-design-system.md  
  06-motion-system.md  
  07-color-extraction-strategy.md  
  08-music-provider-architecture.md  
  09-pairing-engine.md  
  10-security-privacy.md  
  11-accessibility.md  
  12-analytics-observability.md  
  13-testing-strategy.md  
  14-roadmap.md  
  15-risk-register.md  
  16-implementation-backlog.md  
  17-definition-of-done.md
```

Keep these documents specific to the actual repository.

Do not produce generic filler documentation.

Phase 2 — Foundation implementation

After the plan is documented:

1. Initialize or cleanly configure the workspace.
2. Add only justified dependencies.
3. Configure:
4. TypeScript
5. Linting
6. Formatting
7. Testing
8. Environment validation
9. Build scripts
10. CI checks
11. Create shared packages.
12. Implement the initial domain schemas.
13. Implement design tokens.
14. Implement foundational mobile components.
15. Implement initial navigation.
16. Implement typed provider boundaries.
17. Add tests for all foundational logic.

Phase 3 — First vertical slice

Build one complete vertical slice before expanding broadly.

The first vertical slice should be:

1. Launch app.
2. Complete minimal onboarding.
3. Open Capture.
4. Select an image from the photo library.
5. Extract a palette locally.
6. Display animated palette results.
7. Allow palette confirmation.
8. Select a mock music pairing.
9. Save the Memory locally.
10. Display it in the personal timeline.
11. Open the Memory detail page.
12. Handle loading, error, empty, and permission states.
13. Include tests for the core flow.

Use a mock music provider during this slice.

Do not block the first slice on Spotify credentials.

Phase 4 — Real service integration

Once the vertical slice is stable:

- Connect Supabase
- Add authentication
- Add protected media storage
- Add Spotify
- Add cloud synchronization
- Add analytics
- Add subscriptions

21. Initial screen inventory

At minimum, plan these mobile screens:

- Splash
- Welcome
- Onboarding introduction
- Permission education
- Sign in
- Create account
- Home
- Capture
- Photo preview
- Palette extraction progress
- Palette result
- Palette editor
- Pairing recommendations
- Track search
- Track preview
- Memory composer
- Memory saved confirmation
- Timeline
- Timeline filters
- Memory detail
- Collections
- Collection detail
- Discovery
- Discovery filters
- Public Memory detail
- Share composer
- Profile
- Music connection settings

- Subscription and paywall
- General settings
- Privacy settings
- Export settings
- Empty states
- Error states
- Offline state

Do not implement all screens immediately.

Prioritize the vertical slice.

22. Design system foundation

Create semantic tokens rather than scattering raw values.

Include:

- Color primitives
- Neutral surfaces
- Text colors
- Semantic states
- Palette-derived accents
- Typography
- Spacing
- Radius
- Borders
- Shadows
- Blur
- Motion duration
- Motion easing
- Spring presets
- Opacity
- Layering
- Icon sizing
- Touch targets

Define clear rules for dynamically applying a user palette.

For example:

- Generate safe foreground colors
- Clamp saturation when used as large surfaces
- Preserve neutral content surfaces
- Calculate contrast
- Provide fallback accents
- Avoid using extracted colors directly for body text

- Support light and dark visual contexts

23. Initial component foundation

Create a small but high-quality foundational component set.

Potential components:

- Screen
- Text
- Heading
- Button
- IconButton
- Pressable
- Surface
- Input
- SearchInput
- Chip
- PaletteStrip
- PaletteSwatch
- MemoryCard
- TrackCard
- Waveform
- EmptyState
- ErrorState
- LoadingState
- PermissionState
- BottomSheet
- Modal
- Toast
- AudioPreviewControl

Do not create a huge speculative UI library.

Build components when they are required by actual screens.

24. Code quality rules

Mandatory rules:

- Strict TypeScript
- Avoid `any`
- Avoid large god components
- Avoid business logic in presentation components
- Avoid direct Supabase calls inside screen components
- Avoid direct Spotify response types in domain code

- Avoid raw colors and spacing inside feature screens
- Avoid untyped analytics events
- Avoid global state for local UI concerns
- Avoid unnecessary abstractions
- Avoid premature optimization
- Avoid placeholder components being presented as completed features
- Avoid silent error swallowing
- Avoid massive index barrel files that create cycles
- Avoid deeply nested relative imports where aliases are appropriate

Prefer:

- Feature-oriented modules
- Dependency inversion around external services
- Small composable functions
- Explicit domain models
- Testable pure logic
- Clear loading and error states
- Measured performance decisions

25. Performance requirements

Pay special attention to:

- Large images
- Color extraction CPU usage
- Timeline virtualization
- Image caching
- Animated gradients
- Waveform rendering
- Audio preview lifecycle
- Memory leaks
- Reanimated worklets
- App startup time
- Upload compression
- Thumbnail generation
- Query cache size
- Unnecessary rerenders

Use:

- Proper image resizing
- Thumbnail variants
- List virtualization
- Memoization only where measured or clearly justified
- Native-thread animations
- Background processing where supported

- Cancellation for abandoned operations

Document performance budgets.

26. Database and security requirements

When using Supabase:

- Use migrations
- Enable RLS on all user-owned tables
- Create explicit policies
- Separate private and public records
- Do not rely on client-side checks alone
- Use storage policies
- Validate inputs server-side
- Avoid exposing provider tokens
- Encrypt or securely store refresh credentials
- Use Edge Functions for secrets and privileged operations
- Add indexes for timeline and discovery queries
- Add soft-delete or archival strategy where appropriate

Create a seed strategy for local development.

27. Environment configuration

Create an environment validation system.

Document variables such as:

```
EXPO_PUBLIC_SUPABASE_URL
EXPO_PUBLIC_SUPABASE_ANON_KEY
NEXT_PUBLIC_SUPABASE_URL
NEXT_PUBLIC_SUPABASE_ANON_KEY
SPOTIFY_CLIENT_ID
SPOTIFY_CLIENT_SECRET
SPOTIFY_REDIRECT_URI
REVENUECAT_IOS_API_KEY
REVENUECAT_ANDROID_API_KEY
SENTRY_DSN
```

Do not commit secrets.

Create:

- `.env.example`
- Environment validation
- Setup documentation
- Safe mock mode when credentials are unavailable

28. Definition of done for the first implementation cycle

The first cycle is complete only when:

- Repository inspection is documented
- Architecture is documented
- Development environment starts successfully
- Type checking passes
- Linting passes
- Tests pass
- Mobile app launches
- Navigation works
- Design tokens are in use
- Basic accessibility labels exist
- Photo selection works
- Palette extraction works on representative images
- Palette animation performs smoothly
- A mock track can be paired
- A Memory can be saved locally
- Timeline displays the saved Memory
- Memory detail opens
- Empty, loading, failure, and permission states exist
- README contains exact setup instructions
- No secrets are committed
- No major placeholder is falsely labeled complete

29. Expected output during execution

At the start, provide:

1. Repository assessment
2. Assumptions
3. Risks
4. Proposed architecture
5. Milestone plan
6. Initial implementation backlog
7. Files that will be created or modified

Then begin implementation.

Do not stop after only providing a plan unless the environment prevents file modification.

After each meaningful implementation phase, report:

- What was implemented
- Files changed
- Commands run
- Checks passed
- Checks failed
- Known limitations
- Next highest-priority task

30. Important execution constraints

- Do not ask broad questions that can be resolved by inspecting the repository.
- Do not claim implementation success without running relevant checks.
- Do not fake integrations when credentials are missing.
- Use typed mock providers where external services are unavailable.
- Do not rewrite the entire repository without understanding it.
- Do not introduce unnecessary dependencies.
- Do not create dozens of empty placeholder files.
- Do not prioritize visual polish over a working architecture, but do not accept a generic or visually weak UI.
- Preserve existing working logic unless a documented architectural reason requires change.
- Prefer completing one polished vertical slice over producing many incomplete screens.
- Treat security, privacy, accessibility, and performance as part of implementation, not future cleanup.
- Ensure the application remains buildable after every phase.

31. Begin now

Start by inspecting the current workspace.

Then:

1. Produce the repository audit.
2. Create the architecture and implementation plan.
3. Initialize the project if required.
4. Build the foundational design and motion systems.
5. Implement the first vertical slice.
6. Run all available checks.
7. Provide an honest completion report.

The expected result is not merely a collection of screens.

It should be the beginning of a distinctive, production-grade creative platform whose technical architecture, interaction design, and visual identity are strong enough to support a real commercial launch.